CS 410 Milestone Report: Text-Based Spam Email Detection

Nathan Xie (nathanx2), Vijwal Akula (vrakula2), Kangyi Jiang (kangyij2), Zaid Khan Mohammed (zmoha7), Brian Cochran (brianac5)
University of Illinois Urbana-Champaign
Champaign, Illinois, USA

Abstract

This milestone report summarizes the initial development and evaluation of our text-based spam email detection system. Thus far, we have implemented a complete data ingestion pipeline, developed a preprocessing module capable of handling raw, multi-part email formats, and conducted comprehensive exploratory data analysis (EDA) to validate the adequacy of our datasets.

To establish performance baselines, we implemented two text representation methods: a TF-IDF vectorizer and a Word2Vec-based embedding approach, both paired with a Logistic Regression classifier. Training these models on the corporate Enron dataset and evaluating them on the public Apache SpamAssassin corpus yielded overall accuracies of 64.16% for TF-IDF and 62.75% for Word2Vec. While the baseline TF-IDF model failed to correctly identify spam instances in the test set, the Word2Vec representation demonstrated a modest improvement by successfully detecting a small number of spam emails.

These results highlight the challenges of domain shift and feature sparsity in cross-corpus evaluation and suggest that semantic representations alone are insufficient for robust generalization. Our findings reinforce the need to incorporate structural and metadata-based features to build a more reliable and adaptable spam detection system.

Keywords

Spam Detection, Natural Language Processing, TF-IDF, Word2Vec, Logistic Regression, Domain Shift, Feature Engineering, Exploratory Data Analysis

1 Introduction

Despite continuous advancements in automated heuristic filtering, email remains a primary vector for cyberattacks, phishing, and unwanted promotional distribution. Traditional spam filters often rely on static blacklists (e.g., flagging known malicious IP addresses or specific sender domains) or rigid rule-based systems. However, malicious actors rapidly adapt to these defenses by rotating domains and utilizing sophisticated social engineering tactics. Distinguishing between legitimate, opted-in promotional newsletters and malicious phishing attempts now requires a deeper, context-aware analysis of textual and structural features.

The primary objective of our CS 410 final project is to move beyond simple keyword matching and static rulesets. By implementing supervised machine learning techniques and Natural Language Processing (NLP), we aim to build a classifier capable of generalizing the underlying linguistic patterns of spam.

For this milestone, our primary objectives were to establish a functional, end-to-end data processing pipeline, validate the adequacy of our datasets through Exploratory Data Analysis (EDA), and evaluate a baseline classification model. By analyzing the performance—and specifically the failure points—of a standard TF-IDF and Logistic Regression pipeline, we establish a foundational baseline. Understanding exactly why this simple baseline fails allows us to quantify the linguistic and structural gaps that our final, multi-faceted models will address in the latter half of the semester.

2 Methodology and Pipeline

Our current software architecture is implemented entirely in Python. We leverage the Scikit-learn library for machine learning algorithms and metric evaluation, and Pandas for robust dataframe manipulation. The pipeline is structured into three sequential stages: data loading, text preprocessing, and model training using both TF-IDF and Word2Vec feature representations.

2.1 Dataset Selection and Rationale

A machine learning model is only as robust as its training data. For this project, we explicitly chose to utilize two distinct, publicly available datasets to test our model's ability to generalize across different environments.

For model training, we rely on the Enron Spam Dataset. This corpus is heavily skewed toward corporate communications, internal memos, and associated spam from the early 2000s. For testing and evaluation, we utilize the Apache SpamAssassin Public Corpus. Unlike the Enron data, the SpamAssassin dataset represents general, wild internet email traffic.

By training a model on internal corporate data and forcing it to predict classifications on a broad public dataset, we simulate a challenging real-world scenario: evaluating how a localized model reacts to Out-of-Distribution (OOD) data.

2.2 Data Ingestion and Preprocessing

Raw email data inherently contains significant structural noise, making preprocessing a critical and complex component of our pipeline. The Enron dataset was provided in a relatively clean CSV format; however, the SpamAssassin data consists of raw, multipart '.eml' files containing headers, boundaries, and mixed MIME types.

To ingest this data, we engineered a custom recursive parser. This parser traverses the nested MIME structures of the raw files to locate and extract plain text payloads. When emails are formatted exclusively as HTML (a common tactic in marketing spam), the parser seamlessly falls back to extracting the HTML payload for further cleaning.

Following payload extraction, our preprocessing module applies a sequence of targeted regular expressions to normalize the text space:

(1) **HTML Stripping:** Residual HTML tags are removed, leaving only the visible text.

(2) **URL Tokenization:** We replace explicit hyperlinks with a generic "URL" token. We explicitly chose to tokenize rather than delete these links because the high frequency of hyperlinks is a strong structural indicator of spam.
(3) **Noise Reduction:** We remove non-essential punctuation and special characters while retaining basic sentence structure delimiters.
(4) **Normalization:** All text is cast to lowercase to dramatically reduce the dimensionality of our feature space (e.g., ensuring "Free" and "free" map to the same token).

Finally, the email subject line is concatenated with the body text to form a single, unified feature string, ensuring that high-value context from the subject line is preserved for the vectorizer.

2.3 Baseline TF-IDF, Word2Vec, and Logistic Regression

To construct our initial baseline models, we utilized two text representation methods: Term Frequency-Inverse Document Frequency (TF-IDF) and Word2Vec. In basic Bag-of-Words models, frequent English words like "the" or "and" mathematically overwhelm the model, obscuring meaningful terms. TF-IDF solves this by evaluating the relative importance of a term. It balances the term's local frequency within a specific email (TF) against its global rarity across the entire corpus (IDF). This mathematically down-weights common stop-words while assigning high weights to discriminative terms specific to spam or ham.

For this milestone, our TF-IDF vectorizer extracts only unigrams (single words). These numerical feature vectors are then passed to a Logistic Regression classifier. We selected Logistic Regression for our baseline due to its computational efficiency, interpretability, and historical reliability in drawing linear decision boundaries in high-dimensional text classification tasks.

In addition to the TF-IDF baseline, we implemented a Word2Vec-based representation to evaluate whether semantic embeddings could improve generalization across datasets. Word2Vec learns dense vector representations of words based on their surrounding context, enabling the model to capture semantic similarity between terms rather than relying solely on exact word matching. The resulting document-level vectors were also used as input features for the Logistic Regression classifier, allowing for a direct comparison between lexical and semantic feature representations.

3 Exploratory Data Analysis (EDA)

Prior to model training, we conducted an exploratory data analysis on the aggregated datasets to confirm data adequacy, verify the effectiveness of our cleaning scripts, and uncover underlying structural patterns. We successfully processed 33,716 training emails (Enron) and 3,898 testing emails (SpamAssassin).

3.1 Class Distribution

As visualized in Figure 1, our aggregated dataset exhibits a natural class imbalance, with a higher proportion of legitimate "Ham" emails (19,046 instances) compared to "Spam" (18,568 instances). However, the presence of over 18,000 spam examples provides a sufficiently robust foundation for supervised learning. This volume confirms that our dataset is highly adequate for the scope of this final project without requiring synthetic oversampling (like SMOTE) at this stage.

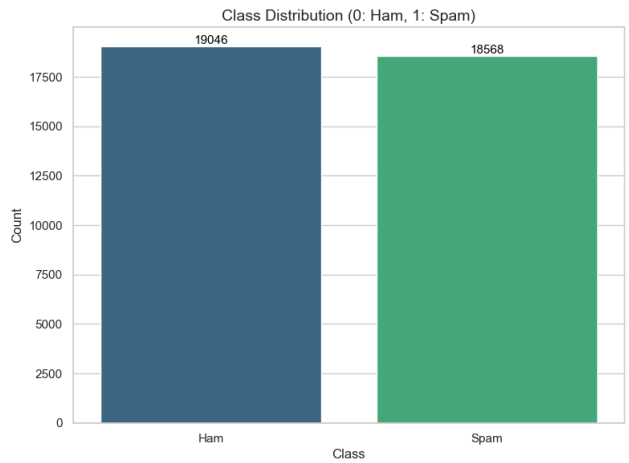

Figure 1: Class distribution of Ham vs. Spam across the combined Enron and SpamAssassin datasets.

3.2 Text Length Analysis

We analyzed the verbosity of emails across the two classes. As demonstrated in the distribution plot (Figure 2), there are observable differences in the total word counts between legitimate correspondence and spam.

Spam messages frequently exhibit a more constrained, highly concentrated length distribution. This is likely due to spam campaigns utilizing automated, templated scripts designed to convey a message quickly before user attrition. Conversely, human-typed ham emails show a much wider variance in length, ranging from one-word replies to multi-page memos. This structural disparity strongly indicates that raw character density and word counts will serve as valuable non-lexical features in our final model iteration.

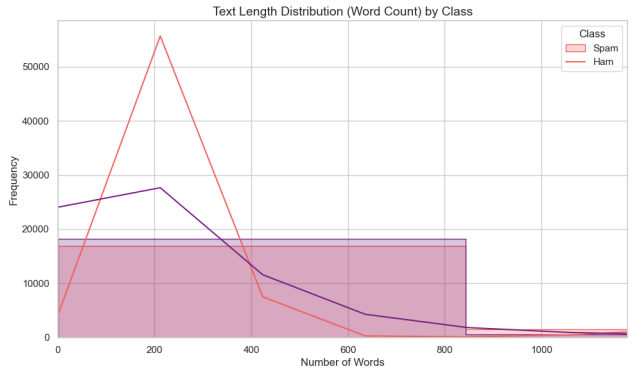

Figure 2: Comparison of text length distribution (word count) between the ham and spam classes.

3.3 Lexical Analysis

Finally, we analyzed the most frequent non-stopword terms within our preprocessed text to verify our cleaning methodology. Figure **??** highlights a stark divergence in vocabulary. The spam corpus is heavily populated with urgent and promotional markers. In contrast, the Enron ham corpus is heavily dominated by corporate, operational, and organizational terminology specific to a large energy firm.

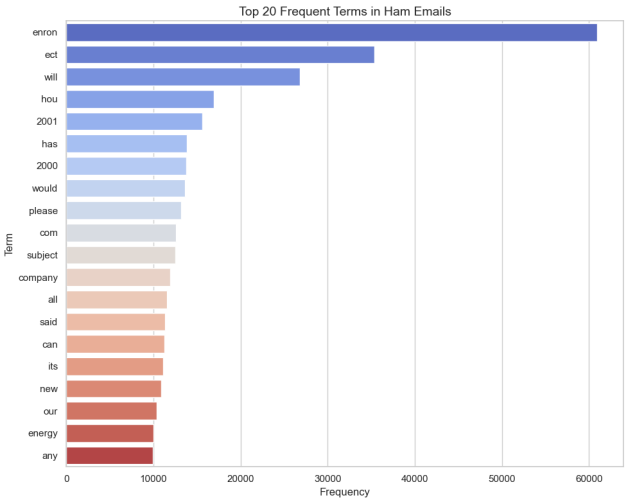

Figure 3: Top terms in ham emails.

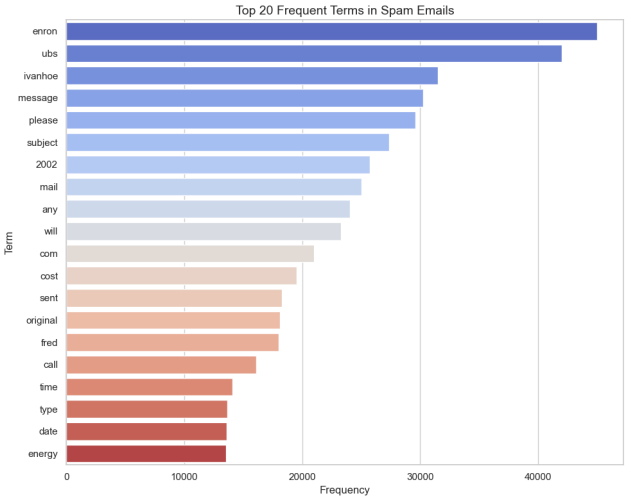

Figure 4: Top terms in spam emails.

4 Baseline Evaluation and Insights

4.1 Performance Metrics

The baseline Logistic Regression model achieved an overall accuracy of 64.16% on the SpamAssassin test set. While this metric might initially appear moderate, accuracy is notoriously misleading in binary classification tasks with skewed testing distributions. A granular review of the precision and recall metrics exposes a critical performance failure.

The model recorded 0.00 precision and 0.00 recall for the Spam class. As detailed in the confusion matrix (Figure 5), the classifier successfully identified all 2,501 legitimate emails (True Negatives) but misclassified all 1,397 spam emails (False Negatives). In practical terms, the model simply defaulted to predicting the majority class ("Ham") for every single instance in the testing set.

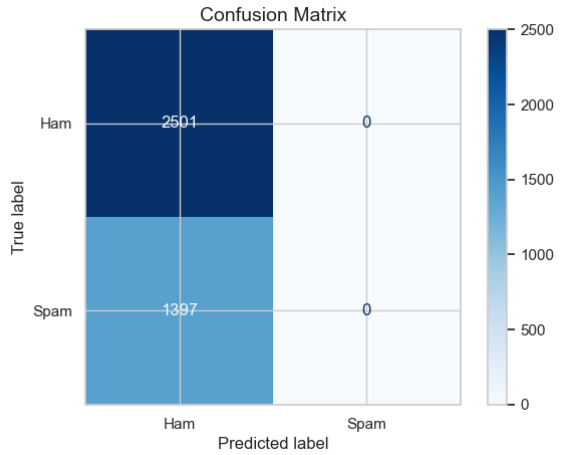

Figure 5: Confusion Matrix of the Baseline Logistic Regression Model on the SpamAssassin Test Set.

4.2 Word2Vec Model Evaluation

To evaluate whether semantic representations could improve classification performance, we implemented a Word2Vec-based feature extraction method. Unlike TF-IDF, which relies on exact vocabulary matching, Word2Vec generates dense vector embeddings that capture semantic similarity between words.

As shown in Figure 6, the Word2Vec model achieved a modest improvement over the TF-IDF baseline. The classifier successfully identified 18 spam emails that were previously misclassified by the TF-IDF model, indicating that semantic representations can partially mitigate feature sparsity caused by domain shift.

However, the model still misclassified the majority of spam emails (1,379 instances) as ham, resulting in extremely low recall for the spam class. These results suggest that improving feature representation alone is insufficient to address the domain shift between the Enron training dataset and the SpamAssassin testing dataset. Additional structural and contextual features will be necessary to significantly improve spam detection performance.

4.3 Discussion: Feature Sparsity and Domain Shift

The severe underperformance of our baseline model provides a highly valuable analytical insight into the limitations of purely lexical models. The total failure to identify spam is a textbook

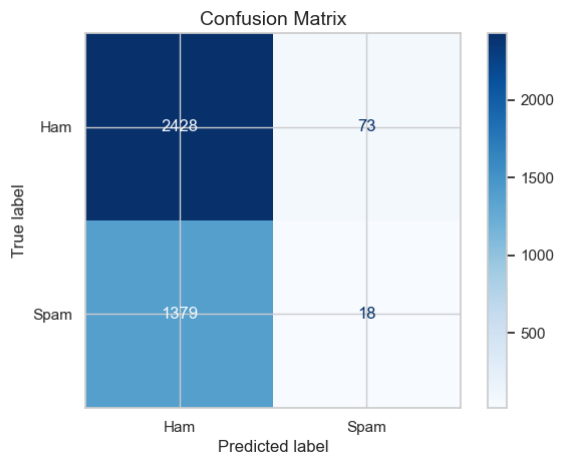

Figure 6: Confusion Matrix of the Word2Vec-Based Classification Model on the SpamAssassin Test Set.

demonstration of "domain shift" causing feature sparsity during inference.

Because the model was trained exclusively on Enron data, the Logistic Regression classifier optimized its decision boundary based on the specific vocabulary of early-2000s corporate spam. However, the SpamAssassin test set represents generalized public internet traffic. The overlapping vocabulary between Enron corporate spam and public internet spam was minimal.

When the Logistic Regression classifier processed the SpamAssassin test data, the input feature vectors were incredibly sparse. The model simply did not see the specific high-weight unigrams it had learned to associate with the positive class (Spam). Lacking the necessary feature activations to push the probability output above the 0.5 decision threshold, the model defaulted entirely to predicting the negative class (Ham).

5 Future Work and Next Steps

This milestone result empirically validates the core hypothesis outlined in our initial project proposal: vocabulary-based keyword matching is inherently brittle and insufficient for robust spam detection across diverse environments.

Moving into Weeks 7 and 8 of our project timeline, our primary objective is to engineer a model that is resistant to domain shift. We will execute the following steps:

- **Structural Feature Engineering:** We will expand our feature extraction pipeline beyond TF-IDF to include non-lexical, structural metadata. By calculating the frequency of URLs, the ratio of capitalized letters to lowercase letters, and overall punctuation density, we aim to capture the mathematical "shape" of a spam email independently of its specific vocabulary.
- **N-Gram Contextualization:** We will upgrade our TF-IDF vectorizer to capture bigrams and potentially trigrams, allowing the model to interpret phrase-level context rather than relying solely on isolated, sparse unigrams.
- **Comparative Modeling:** We will implement a Linear Support Vector Machine (SVM) and conduct hyperparameter tuning (e.g., adjusting regularization strengths) to benchmark its performance on this new feature space against our updated Logistic Regression model.

By integrating these structural features and experimenting with models better suited to finding maximum margins in high-dimensional spaces, we anticipate a significant increase in recall, moving us closer to a generalized, production-ready spam detection system.

6 Code Availability

The source code for this project, including data preprocessing, model training, and evaluation scripts, is publicly available at the following GitHub repository:

https://github.com/zaidmohammed7/CS-410-Final-Project-Team-18-SP26

The repository contains instructions for reproducing the experiments and generating the results presented in this report.